STC FRAMEWORK

Stalwart · Trainer · Critic

Architecture, Design & Capabilities

An Open-Source Framework for Production-Grade AI Agent Systems
with Built-In Optimization, Zero-Trust Governance,
Data Sovereignty, and AIUC-1 Compliance Alignment

Author: Nitesh Ambastha

Version 1.0 — March 2026
https://github.com/nitesh71078/stc-framework

Table of Contents

1. Executive Summary

The STC Framework is an open-source architectural pattern for building production-grade AI agent systems. It brings software engineering discipline to AI agents by structurally separating execution, optimization, and governance into distinct system roles.

Most AI agent frameworks focus on making agents smarter. STC focuses on making agent systems more disciplined, auditable, and production-ready. The framework is designed so that a properly configured deployment satisfies AIUC-1 certification requirements — the world's first AI agent standard.

1.1 Core Thesis

An AI agent should be designed as a system of distinct personas, each with a single responsibility and clear separation of concerns. Not peers. Not collaborators. System roles with asymmetric authority.

1.2 The Three Personas

Persona	Role	Responsibility
S — Stalwart	Execution Plane	Performs business tasks. Optimized to act — not to judge itself, retrain itself, or trust itself.
T — Trainer	Optimization & Control Plane	Makes the Stalwart better over time. Monitors performance, optimizes cost, tunes prompts, selects models.
C — Critic	Zero-Trust Governance Plane	Assumes nothing is trustworthy by default. Verifies outputs, detects hallucinations, enforces compliance.

1.3 Supporting Architectural Layers

Layer	Role	Key Principle
Sentinel Layer	Infrastructure Enforcement	Enforces trust boundaries, routing, auth, PII redaction. Infrastructure, not intelligence — does not learn.
Declarative Specification	System Contract	Versioned YAML defining all policies, thresholds, and compliance mappings. Single source of truth.
Observability	Unified Audit	Correlated traces across all components. Immutable audit trail. AIUC-1 evidence generation.
Operational Control	FinOps & Cost Management	Token governance, rate limits, burst control, cost circuit breakers, per-persona budget allocation.

2. System Architecture

The STC Framework is organized in concentric defense layers, with the agent execution at the center and security perimeters radiating outward.

2.1 Defense-in-Depth Architecture

Five independent security and governance layers, each catching what the others miss:

```
Layer 1 (Outermost): Runtime AI Firewall (LlamaFirewall)
  │  ML-based jailbreak detection, chain-of-thought auditing
  │
Layer 2: Gateway (Kong / Bifrost / LiteLLM)
  │  LLM routing, MCP governance, tool access control
  │
Layer 3: Auth & Identity (Casbin RBAC)
  │  Persona permissions, operator roles, API key lifecycle
  │
Layer 4: Data Security (Presidio + DLP)
  │  PII surrogate tokenization, prompt injection, usage policy
  │
Layer 5 (Innermost): Domain Guardrails (NeMo + Guardrails AI)
     Hallucination detection, numerical accuracy, scope enforcement
```

2.2 Request Flow

Every user request flows through the full defense stack:

```
User Query
  ↓
[Runtime Firewall] → Block if jailbreak/injection detected
  ↓
[Auth Check] → Verify persona identity and permissions
  ↓
[Data Security] → Mask PII, check DLP, enforce usage policy
  ↓
[Cost Control] → Pre-request budget check, token governance
  ↓
[MCP Catalog] → Validate tool access against registry
  ↓
[Gateway] → Route to LLM based on data tier classification
  ↓
[STALWART] → Execute: retrieve → reason → respond
  ↓
[CRITIC] → Evaluate: guardrails → verdict → escalation
```

```
  ↓
[Runtime Firewall] → Output alignment check
  ↓
[Data Security] → Reverse surrogate tokens for user
  ↓
[TRAINER] → Record trace, compute reward, optimize
  ↓
[Observability] → Correlated audit event
  ↓
Response to User
```

3. Sentinel Layer — Modules

The Sentinel Layer is the infrastructure enforcement surface of STC. It contains seven modules, each addressing a specific enterprise security and operational concern.

3.1 Runtime AI Firewall

Module: sentinel/runtime_firewall.py

Engine: Meta LlamaFirewall (MIT License)

The outermost security perimeter. Operates as a system-level defense layer independent of application code. Even if guardrails are misconfigured, the firewall catches threats.

Scanner	What It Detects	Performance
PromptGuard 2	Direct jailbreak attempts, prompt injection	97.5% recall, 1% FPR, 19.3ms latency
AlignmentCheck	Chain-of-thought hijacking, goal misalignment	83% attack reduction, audits agent reasoning
CodeShield	Insecure code generation, vulnerability patterns	Real-time static analysis
Custom Scanners	Regex and LLM-based, configured from spec	User-defined patterns

Fallback: When LlamaFirewall is not installed (no GPU), the system falls back to regex-based detection with reduced coverage.

3.2 Gateway Adapters

Module: sentinel/adapters.py

Gateway-agnostic interface supporting three backends, switchable via a single spec field:

Gateway	LLM Routing	MCP Governance	Tool Access	Overhead	Best For
LiteLLM	100+ providers	No	Advisory only	~8ms	Development
Kong AI	10+ providers	Full (OAuth 2.1)	Registry + ACL	2-5ms	Enterprise
Bifrost	20+ providers	Native MCP	Per-team controls	~11µs	Performance

```
sentinel.gateway.engine: kong  # litellm | kong | bifrost
```

3.3 Authentication & Authorization

Module: sentinel/auth.py

Engine: Apache Casbin (Apache 2.0)

Machine-to-Machine (Persona Auth)

Every action in STC goes through a Casbin RBAC check. Permissions are defined in the Declarative Specification and enforced at runtime. This implements the asymmetric authority model: Stalwart cannot modify routing, Trainer cannot trigger escalation, Critic cannot bypass governance.

Human Operator RBAC

Role	Modify Spec	Reset Escalation	Approve Routing	Export Audit	Rotate Keys
Admin	Yes	Yes	Yes	Yes	Yes
Operator	No	Yes	Yes	No	Yes
Auditor	No	No	No	Yes	No
Viewer	No	No	No	No	No

API Key Management

Cryptographically generated keys, stored as SHA-256 hashes, scoped per persona, with configurable rotation (default 90 days) and revocation with audit trail. Supports JWT validation for IdP integration (Okta, Azure AD, Auth0, KeyCloak).

3.4 Data Security

Module: sentinel/data_security.py

Engines: Microsoft Presidio (MIT) + custom DLP patterns

Capability	What It Does	AIUC-1 Mapping
PII Surrogate Tokenization	Replaces PII with reversible tokens before LLM call. LLM never sees real data. Tokens reversed on output.	A006
Prompt Injection Defense	Pattern-based + encoding detection for OWASP LLM Top 10 injection techniques	F001
Usage Policy Enforcement	Blocks investment advice, out-of-scope content based on risk taxonomy	C001
Data Loss Prevention	Scans all output channels (responses, logs, traces)	A004

Capability	What It Does	AIUC-1 Mapping
	for proprietary data leakage	
Content Safety	Toxicity detection, harmful content filtering	C - Safety

3.5 MCP Catalog Integration

Module: sentinel/mcp_catalog.py

Connects STC to enterprise MCP registries for runtime tool discovery, health monitoring, and governance validation. Adapts automatically based on gateway: spec-based (default), Kong MCP Registry, or Bifrost tool registry.

Capability	Description
Tool Discovery	Query available MCP servers from registry, match against permitted_tools in spec
Health Monitoring	Periodic health checks; mark unhealthy tools as degraded
Governance Validation	Verify every tool call is registered; detect shadow tools (called but not registered)
Catalog Sync	Compare registry vs spec: new tools, missing tools, shadow tools, unhealthy tools
Trainer Integration	Expose tool metrics (latency, reliability) for Trainer optimization

4. Core Personas

4.1 Stalwart (Execution Plane)

Module: stalwart/financial_qa_agent.py

Framework: LangGraph (MIT)

The Stalwart executes business tasks. It does NOT evaluate its own accuracy, choose its own guardrails, optimize its own prompts, or decide if it is safe to respond. Those responsibilities belong to the Trainer and Critic.

Reference Implementation: Financial Document Q&A

LangGraph RAG workflow: classify query → retrieve from local vector store → assemble context → call LLM via Sentinel gateway → format cited response. All retrieval and embedding happens locally (data sovereignty). The LLM is routed based on data classification tier.

4.2 Trainer (Optimization & Control Plane)

Module: trainer/optimization_manager.py

Engine: Microsoft Agent Lightning (MIT) + LiteLLM

The Trainer makes the Stalwart better over time through three optimization loops:

Optimization Loop	What It Adjusts	Signal Source
Retrieval Optimization	Chunk size, top-k, reranking weights	Retrieval quality scores from traces
Prompt Optimization	System prompt, few-shot examples via Agent Lightning APO	Factual accuracy + user feedback
Model Routing	Which model handles which query type (cost vs accuracy)	Cost-normalized accuracy per model

Reward Computation

Three weighted reward signals: user feedback (40%), factual accuracy (40%), retrieval quality (20%). These feed into Agent Lightning's RL optimization loop (GRPO/PPO) for model weight updates and prompt refinement.

Maintenance Mode

When the Trainer detects suboptimal performance (accuracy below 85%, cost above $0.10/task, hallucination rate above 5%, P95 latency above 5000ms), it can place the Stalwart into maintenance mode: degrade, pause, or alert-only per spec configuration.

4.3 Critic (Zero-Trust Governance Plane)

Module: critic/governance_engine.py

Engines: NVIDIA NeMo Guardrails (Apache 2.0) + Guardrails AI Hub (Apache 2.0)

The Critic assumes nothing is trustworthy. It does not fix problems — it submits structured feedback to the Trainer and triggers escalation when thresholds are breached.

Guardrail Pipeline

Guardrail	What It Catches	Severity	Action
Numerical Accuracy	Numbers in response not grounded in source documents	Critical	Block
Hallucination Detection	Claims not supported by retrieved context	Critical	Block
PII Output Scan	PII in LLM-generated response	Critical	Block
Investment Advice	Buy/sell/hold recommendations, portfolio advice	High	Block
Scope Check	Off-topic responses outside financial document Q&A	Low	Warn

Graduated Escalation

Level	Trigger	Action
Degraded Mode	2 critical failures in 10 tasks	Add disclaimers, require source citations
Quarantine Mode	3 critical failures in 10 tasks	Hold responses for human review before delivery
Suspension	5 critical failures in 10 tasks	Suspend all personas, require human reset
Circuit Breaker	3 consecutive failures	300-second cooldown with auto-retry

5. Observability & Audit

Module: observability/stc_observability.py

Engines: OpenTelemetry + Arize Phoenix + Langfuse

5.1 Unified Trace Context

Every request generates a correlated trace spanning all components: firewall → auth → sentinel → stalwart → critic → trainer. A single trace ID links every event, enabling end-to-end auditability.

5.2 Immutable Audit Trail

Events are append-only and cannot be modified after creation. Each event carries: timestamp, component, event type, severity, action, outcome, evidence, and AIUC-1 requirement mapping. Daily JSONL rotation with Parquet export for long-term compliance retention (configurable, default 365 days).

5.3 AIUC-1 Compliance Evidence Generator

Generates structured evidence packages for AIUC-1 audits across all six domains: Data & Privacy, Security, Safety, Reliability, Accountability, Society. An auditor gets machine-readable reports mapping every requirement to STC controls and evidence.

5.4 Log Sanitization

DLP-aware trace processor that redacts proprietary data before export to external observability platforms (Phoenix, Langfuse, Datadog). Ensures observability channels do not become data leak vectors.

5.5 Prompt Registry

Engine: Langfuse (MIT, self-hosted)

Centralized prompt management with automatic versioning, environment labels (dev/staging/production), and trace linkage. Every Stalwart execution records which prompt version was active. The Trainer registers new prompt versions when it optimizes, creating full prompt lineage for audit.

6. Operational Control & Cost Management

Module: operational/cost_controls.py

Enterprise FinOps controls that prevent cost overruns and resource exhaustion in production AI agent systems.

Control	What It Does	Example
Token Governor	Max input/output/context tokens per request	50K input tokens → rejected before LLM call
Rate Limiter	Per-persona RPM/RPH with concurrent caps	Stalwart: 30 RPM, Trainer: 10 RPM
Burst Controller	Runaway loop detection, max LLM calls per workflow	Agent stuck in retry → circuit breaker after 10 calls
Cost Circuit Breaker	Pre-request budget check with graduated response	80% warn → 90% throttle → 95% pause → 100% halt
Anomaly Detector	Alert when cost exceeds N× rolling average	$0.35 request when avg is $0.008 → anomaly alert
Quota Management	Per-persona budget allocation from daily total	Stalwart 80%, Trainer 15%, Critic 5%

All cost controls operate BEFORE the LLM call (not after). The system knows the estimated cost before committing spend, and takes graduated action: warn → throttle → pause → halt.

7. Data Sovereignty

Proprietary firm data never leaves the trust boundary. This is an architectural principle, not a configuration option.

7.1 Tiered Data Classification

Tier	Routing Policy	Example Data
Public	Any provider (OpenAI, Anthropic, Bedrock, local)	General market data, public regulatory text
Internal	VPC-only (Bedrock, self-hosted models)	Internal process documents, non-client operational data
Restricted	Local models ONLY (never leaves boundary)	Client PII, account positions, trade data, advisor compensation

7.2 Surrogate Tokenization

Before sending to an external LLM, detected entities are replaced with deterministic surrogate tokens. The LLM reasons over surrogates (e.g., [PERSON_ED2248]) without knowing real values. On response, tokens are reversed for the user. The token map is stored locally, encrypted, and never leaves the boundary.

7.3 Local Infrastructure

Component	Tool	Purpose
Vector Store	Qdrant (Apache 2.0)	Local document embeddings, never sent externally
Embedding Model	BGE / E5 via Ollama (MIT)	Local embedding computation
Local LLM	Ollama / vLLM (MIT / Apache 2.0)	Inference for restricted-tier data

8. AIUC-1 Compliance Alignment

AIUC-1 is the world's first AI agent standard, covering six enterprise risk domains. STC is designed so that a properly configured deployment satisfies AIUC-1 certification requirements out of the box.

AIUC-1 Domain	STC Coverage	Key Modules
A. Data & Privacy	PII protection, IP protection, cross-customer isolation, data boundary enforcement	data_security, gateway, auth
B. Security	Adversarial robustness, prompt injection defense, access controls	runtime_firewall, auth, adversarial testing
C. Safety	Risk taxonomy, pre-deployment testing, harmful output prevention, hallucination detection	critic, guardrails, evaluation suite
D. Reliability	Predictable behavior, error handling, graduated escalation, optimization	trainer, critic escalation, circuit breaker
E. Accountability	Transparency documentation, immutable audit trail, human oversight, prompt versioning	observability, spec, Langfuse
F. Society	Prevent AI misuse, scope enforcement	runtime_firewall, guardrails

8.1 Declarative Specification as Compliance Artifact

The Declarative Specification (stc-spec.yaml) serves as the primary governance document for AIUC-1 audits. It contains a compliance section that explicitly maps each AIUC-1 requirement ID to the STC component that satisfies it and the evidence source. An accredited auditor (e.g., Schellman) reviews this file alongside the audit trail exports.

9. Open-Source Technology Stack

Capability	Tool	License
Agent Execution	LangGraph	MIT
RL-based Optimization	Microsoft Agent Lightning	MIT
LLM Gateway (Default)	LiteLLM	Apache 2.0
LLM + MCP Gateway (Enterprise)	Kong AI Gateway	Apache 2.0
LLM + MCP Gateway (Performance)	Bifrost	Apache 2.0
Runtime AI Firewall	Meta LlamaFirewall	MIT
Guardrail Orchestration	NVIDIA NeMo Guardrails	Apache 2.0
Composable Validators	Guardrails AI + Hub	Apache 2.0
PII Detection + Redaction	Microsoft Presidio	MIT
RBAC Policy Engine	Apache Casbin	Apache 2.0
Prompt Registry + Versioning	Langfuse	MIT
Observability + Audit	OpenTelemetry + Arize Phoenix	Apache 2.0
Local Vector Store	Qdrant	Apache 2.0
Local Embeddings + LLMs	Ollama	MIT
Adversarial Testing	MITRE ATLAS-informed probes	Custom

Everything is self-hostable with no vendor lock-in. The Declarative Specification ties all components together as a single source of truth.

10. Getting Started

10.1 Quick Start

```
git clone https://github.com/nitesh71078/stc-framework.git
cd stc-framework
cp .env.example .env  # Configure API keys
docker-compose up -d  # Start infrastructure
python reference-impl/scripts/load_documents.py
python reference-impl/scripts/run_agent.py
```

10.2 Enterprise Deployment (Kong)

```
docker-compose -f docker-compose.yaml -f docker-compose.kong.yaml up -d
```

Update stc-spec.yaml: sentinel.gateway.engine: kong

10.3 Run the Learning Demo

```
python reference-impl/scripts/learning_demo.py
```

Demonstrates 4 rounds of Trainer optimization with measurable improvement in accuracy and cost.

10.4 Run Adversarial Testing

```
python adversarial/run_red_team.py
```

8 MITRE ATLAS-informed probes testing prompt injection, data exfiltration, hallucination induction, and jailbreak resistance.

— END OF DOCUMENT —